Definições:

- **Node.js (documentação):**
    - É um ambiente de execução, que permite que você execute código JavaScript no seu computador, ou em um servidor. Sem o Node.js o código JavaScript só rodaria em navegadores, ele é responsável por permitir a construção de APIs, servidores e ferramentas de linha de comando.
- **NPM (documentação):**
    - NPM significa: Node Package Manager (Gerenciador de pacotes do Node), ele instala e gerencia pacotes para reutilização. Não há necessidade de reescrever um framework (por exemplo, o Express) do zero toda vez. Você usa o NPM para baixar o trabalho que foi realizado previamente e o disponibilizaram como um pacote.
- **Package.json:**
    - Todo projeto Node.js precisa de um arquivo package.json: Ele guarda o nome do projeto, a versão, scripts, lista de dependências (os pacotes que serão utilizados) necessárias para o projeto e entre outros.
- **Express.js (documentação):**
    - É um framework (conjunto de ferramentas) que facilita a criação de servidores web e APIs com Node.js. Sem o Express.js você tem apenas o Node.js puro, se você quiser criar um servidor precisaria escrever manualmente. O Express.js serve como uma máscara:
        - Deixa a configuração das rotas mais limpas e intuitivas;
        - Checa se a requisição está correta, se o token JWT é válido…
        - Facilita requisições (Request) e respostas (Response).
- **const app = express():**
    - Para todos os efeitos, é o servidor web. É a instância principal da sua aplicação, um objeto gigante com os métodos do Express.
        - app.get('/', ...) | app.post('/pacientes', …) | app.put('/pacientes/:id', …)

Como rodar um projeto do zero com Node.js

Objetivo: Criar um projeto novo em Node.js, instalar o framework Express e rodar nosso primeiro servidor “Olá, Mundo!”

1. **Verificar a existência do Node.js:**
    a. Abra o seu terminal;
    b. Rode o comando:
        i. *node -v*
    c. Se retornar: Número de versão, exemplo: v20.19.1 → Node instalado.
    d. Se retornar: Erro → Node não instalado:
        i. Abra: https://nodejs.org/. Baixe o arquivo *.msi* e instale.
    e. Rode novamente o comando:
        i. *node -v*
2. **Criar a pasta do projeto novo:**
    a. Abra o seu terminal;
    b. Navegue até o diretório/pasta onde você guarda os seus projetos (ex: *Documentos/Projetos)*;
    c. Crie uma nova pasta para o seu projeto (API):
        i. *mkdir minha-primeira-api*
    d. Entre na pasta criada:
        i. *cd minha-primeira-api*
    e. Rode o comando:
        i. *code .*
3. **Iniciar o projeto (gerar o package.json):**
    a. Para que serve o package.json?
        i. Gerenciamento de dependências: Registra quais pacotes seu projeto precisa para funcionar, separando em:
            1. *dependencies* → pacotes essenciais para o projeto rodar em produção;
            2. *devDependencies* → pacotes usados somente para o desenvolvimento, como formatadores de código ou ferramentas de teste.

ii. O tópico *scripts* permite que você crie atalhos para comandos do terminal, exemplo:

1. node -- watch src/server/app.js → npm run dev.

    a. O comando watch é uma flag nativa do node (a partir da versão 18.11,

    b. Scripts geralmente adicionados:

        i. "scripts": {
            "dev": "node --watch src/server/app.js",
            "start": "node src/server/app.js"
           }

b. No VS Code, abra o terminal e rode o comando:

    i. *npm init -y*

    ii. O *npm init* inicia o projeto, e pede: o nome do projeto, versão e etc. Já o *-y* pula todas as perguntas e aceita as respostas padrão.

4. Instalar o Express:

a. Ainda dentro da pasta criada, no terminal do VS Code, rode o comando:

    i. *npm install express*

    ii. Para instalar outro pacote, basta repetir a base do comando e inserir o nome do pacote: *npm install [nome_do_pacote]*.

    iii. O pacote é inserido na pasta *node_modules*, e o *express* é adicionado em *dependencies* no *package.json*.

5. Criar o arquivo do servidor (“Hello, World!”):

a. Crie um arquivo no VS Code dentro da pasta *minha-primeira-api*:

    i. Nomeie o arquivo como: *app.js*

b. Dentro do arquivo importe o *express:*

    i. `import express from "express";`

c. Crie o *app* (servidor):

    i. `export const app = express();`

    ii. `app.use(express.json());`

d. Defina a porta que o servidor deve conectar:

i. `const port = 3000;`

e. Crie um endpoint de ping, que será utilizado para conferir se a API está conseguindo se comunicar com o servidor web:

i.
```
app.get('/ping', (req, res) => {
  // Endpoint para conferir se a API está funcionando.
  console.log('Alguém acessou o endpoint /ping. Ping!');
  res.status(200).send('Hello, World! Pong!');
});
```

f. Faça a aplicação escutar a porta:

i.
```
app.listen(port, () => {
  console.log(`Servidor rodando e ouvindo na porta ${port}`);
  console.log(`Acesse http://localhost:${port} no seu navegador!`);
});
```

g. No VS Code, abra o terminal e rode o comando:

i. *npm install nodemon -D*

1. Se não tiver a versão do watch.

ii. No *package.json* procurar por *"scripts"*:

1. Criar um script para *"start"* e um para *"dev"* dentro do bloco de *"scripts"*:

a. O script dev abaixo é com o exemplo de uso com o nodemon, pode ser utilizado como listado acima no caso do watch.

b.
```
"scripts": {
  "start": "node app.js",
  "dev": "npx nodemon app.js",
```

iii. Novamente, no terminal do VS Code, rode:

1. *npm run dev*

2. No seu navegador, acesse: http://localhost:3000/ping

Como rodar o projeto existente em Node.js

1. **Se você ainda não estiver com o projeto local, criar uma pasta para clonar o repositório:**

a. Abra o seu terminal;
b. Navegue até o diretório/pasta onde você guarda os seus projetos (ex: *Documentos/Projetos)*;
c. Crie uma nova pasta para o seu projeto (API):
    i. *mkdir minha-primeira-api*
d. Entre na pasta criada:
    i. *cd minha-primeira-api*

2. Clonar repositório alocado no GitHub:

a. Ainda no terminal rode os comandos:
    i. *git clone https://github.com/{seu_usuario}/{nome_projeto}.git*
    ii. *cd {nome_projeto}*
    iii. *code .*
b. No VS Code abra o terminal e execute:
    i. *npm install*
c. Se você usar *.env*:
    i. Procure por um arquivo *.env* no projeto;
    ii. Preencha o *.env* com as suas informações.
d. Abra o arquivo *package.json*:
    i. Verifique como os scripts foram configurados, e rode um deles:
        1. *npm run dev* ou *npm start*